Team Green Plant Monitor: Design Document

Contributors: Tyler Orwa and Catalina E. Alcaraz

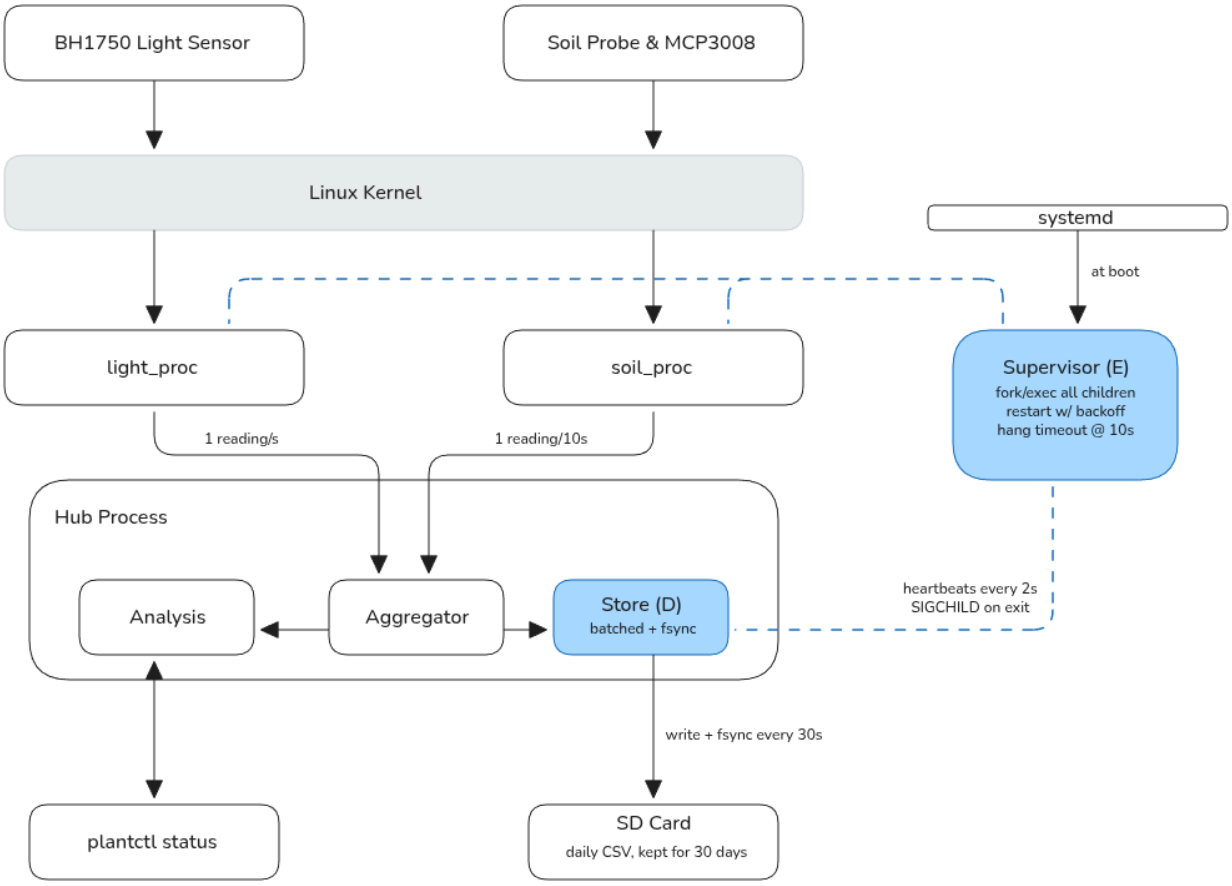
Our team is building an always-on plant monitor utilizing the power of a Raspberry Pi 4. Our idea is for the plant owners who are very busy but are often away for long periods of time or most of the day, these type of plant owners want the satisfaction of a healthy flourishing plant but may find themselves asking, “What is wrong with my plant, and what should I do about it?”. Two sensors, light and soil moisture, run in isolated processes that feed a hub. The hub writes to a crash-safe log where a supervisor keeps everything alive unattended. The analysis code is ours and the device runs with no network connection.

Problem and User

The user is an indoor plant owner who is gone for most of the day. Plants are harmed while unattended: soil dries out on a bright day, stays soggy for a week, or the plant sits in a corner too dark to use the water it is given. A phone reminder can say “Make sure to water your plant on Tuesday at 8am” but it cannot see how fast the pot is actually drying in this room’s light. That takes a device that is present and awake all day, measuring the conditions of the plant.

Our focus is one plant at a time, the device detects three conditions we can induce, and reports how accurately it does. The conditions that could factor into the plant health that we monitor are overwatering, underwatering, and insufficient light.

Architecture



Two sensors enter through separate kernel interfaces, so a fault in one bus can't stall the other. Each sensor process samples on its own timer and sends fixed-size readings to the hub over a Unix domain socket. The hub is a single-threaded loop that checks staleness and sequence gaps, feeds the analysis, and hands readings to the store; the only code writing data to the disk. The supervisor starts every child, restarts any that die or stop sending heartbeats, and is itself restarted by systemd. Between each of them, the two sensors produce a reading each second to not overcrowd the CSV or storage size.

Mechanism Mapping

We implement two mechanisms: Mechanism E (multi-process with a supervisor) and Mechanism D (custom storage layer). Both follow the same idea, no one is required to watch the device when something goes wrong.

Mechanism	Implemented by	Justification	How its measured
E: Multi-Process Architecture	`light_proc` and `soil_proc` (each per sensor), `hub` (aggregates over Unix socket), `supervisor` (exec, signal, restarts, and heartbeat timeouts)	The owner is away for hours so a fault during working hours won't be noticed. A crash or hang must not stop soil readings and must be detected, logged, and recovered without a person for monitoring.	`kill -9` and `SIGSTOP` on each child. Record time to detect, restart, readings lost, and log lines(Evaluation)
D: Custom Storage Layer	`store` (inside the hub): batched-append only, `fsync` policy, recovery on startup	Every diagnosis is a trend of drying rate against light received, gap or corrupted file erases the history. The Pi runs from an SD card and so writes are batched to limit wear.	Batched write rate and `fsync` count per hour. Repeated `kill -9` calls mid-write with validation to each file to measure rows lost (Evaluation)

Failure-Mode Table

Component	**Failure**	**Detected by**	**System response**	**Log shows**
BH1750 (light)	Unplugged	`read` on kernel returns `EIO` or `ENXIO`	Reading sent with `ERROR` flag; backend retries on later samples; diagnoses pause	`LIGHT_READ_ERROR` (rate-limited), `SENSOR_STALE light`, then `SENSOR_RECOVERED light down=Ns`
Soil probe +	Probe unplugged	Values stuck or	Reading flagged	`SENSOR_SUS

MCP3008		identical for N samples	`ERROR`; diagnoses pause	PECT soil` then `SENSOR_REC OVERED soil`
`light_proc` / `soil_proc`	Crash	Supervisor: `SIGCHLD` through `signalfd` then `waitpid`	Restart with increasing backoff (1 s doubling up to 60 s); the other sensor keeps recording	`CHILD_KILLE D <proc>`, `CHILD_STAR TED <proc> pid=N`, `SENSOR_STA LE`/`SENSOR_ RECOVERED` from hub
`light_proc` / `soil_proc`	Hang	Supervisor: heartbeat timeout	`SIGKILL`, then the normal restart path	`CHILD_HUNG <proc>`, then the crash lines above
`light_proc` / `soil_proc`	Crash loop	Supervisor: K restarts within a window	Marked `FAILED`; retried only at the 60 s maximum delay	`CHILD_FAILE D <proc> restarts=K`, then one retry line per minute
Hub	Crash	Supervisor: `SIGCHLD`; sensors: send fails	Hub restarted; sensors buffer recent readings and reconnect with backoff; store recovers file on startup	`CHILD_KILLE D hub`, `HUB_LOST`/` HUB_RECONN ECTED` in each sensor and maybe `STORE_RECO VERED`
Hub	Incorrect message format from client	Size, type and sensor-ID checks	Message dropped	`MSG_INVALI D from=<proc>`
Store	Crash mid-flush	Torn last line found at startup	File truncated to last complete row	`STORE_RECO VERED bytes=N`
Store	`write`/`fsync` fails	Return value checked on every call	Keep buffering and retry next flush; if buffer fills, drop oldest	`STORE_WRIT E_ERROR errno=..` and `STORE_DROP

				PED n=..`
Store	SD card full	Retention and log rotation	Oldest daily file deleted at 30 days; logs rotated at 10 MB	`RETENTION_ DELETE file=..`
Supervisor	Crash	Nothing above it	Run the supervisor as a `systemd` service with `restart=always`; on restart it kills leftover children and starts clean	Systemd entry plus `SUPERVISOR_ START` and `CLEANUP_OR PHAN`
Whole Pi	Power loss	Boot	systemd starts the supervisor; store recovery runs	`SUPERVISOR_ START` after a gap in heartbeats; visible in the CSV

Storage & Data

Readings go to a daily, batched append-only CSV and on power cut at most the only thing lost is the last unflushed batch and startup recovery removes any half-written row. Readings from both light and soil processes are logged once per second for light and once per 10 seconds for soil. They are also kept for 30 days, with the oldest daily file deleted after those 30 days. The process logs and heartbeats are logged usually about every minute as single lines with the time, process, and event details.

Constraints & Substitutions

We cut out the original 4 sensors down to 2, which originally included a CO2 and pH sensor as well. This cost us no pH or air-quality diagnosis capabilities. This cut was for cost and

complexity since those sensors can become very expensive, especially if we aim for reliability and the complexity of adding 2 more surfaces to pull data from would overblow the scope of this term project.

Evaluation Plan

We will measure the: sample-to-hub latency (under 10ms), sampling jitter (under 50ms), dropped readings (0 during normal operation), CPU utilization (under 5%), crash recovery (detected in 1s, restarted within 2s; 0 readings lost), hang recovery (detected and restarted within 10s of heartbeat timeout), and crash consistency (all files valid after recovery; never lose more than 1 batch).

Ownership Map

Each partner owns each mechanism and the other partner reviews every change before it merges:

Partner A: Sensors & Supervisor

Partner B: Pipeline & Storage

Shared: Tools, tests, docs

AI-Use Plan

We use Claude Code to plan and write the code, explain kernel interfaces, review diffs, and generate tests. The shipped product calls no model, API, and has no network capabilities. Each partner works on their own Claude session and branch and copies their transcripts at each milestone. Each PROMPTLOG.md will record at least one review of the other partner’s code.